

WayPoint

AI-Powered Intercity Journey Planner & Bus Operations Platform

SE3090 – Software Engineering Frameworks

Assignment 1 – Integrated Full-Stack and Agentic AI Application Development

Sri Lankan Intercity & Tourist-Corridor Transport Domain

Proposal Detail	Value
Assignment	Assignment 1 – Integrated Full-Stack and Agentic AI Application Development
Domain	Sri Lankan intercity and tourist-corridor journey planning, booking and transport operations
Group	Four students; four primary business components
Core value	Plan practical journeys, compare services, book seats and recover intelligently from disruptions
Mandatory stack	ASP.NET Core Web API, PostgreSQL, React, Flutter and Agentic AI

1. Overview

WayPoint is an AI-powered intercity journey planning and bus operations platform designed for Sri Lankan passengers and transport operators. Unlike a conventional bus-booking application that mainly answers “which bus can I book?”, WayPoint focuses on the broader question: “what is the most suitable journey for my travel objective?” The platform combines deterministic route, timetable, fare, seat and operational data with a controlled Agentic AI workflow that can plan and evaluate journey options, handle operational constraints and recommend safe alternatives.

The passenger uses Flutter to enter an origin, destination, travel date/time and preferences such as arrival deadline, direct journey, budget, bus facilities or preferred boarding point. WayPoint retrieves valid services from the shared ASP.NET Core API and constructs feasible journey options. Where appropriate, the Agentic AI subsystem coordinates specialized agents to analyse route compatibility, fleet/seat feasibility, booking options and safety/validation constraints. The passenger can then select and book a suitable service.

React is the operator and management workspace. It supports route and timetable administration, fleet and driver operations, booking oversight, disruption management, journey-planning analytics and controlled human approval. Both applications use the same ASP.NET Core API, PostgreSQL database, identity model and business rules, satisfying the integrated-system requirement.

2. Real-World Problem

Intercity bus travel can require passengers to compare multiple departures, boarding points, fares, travel times and service conditions. A booking-only workflow does not adequately support passengers whose preferred service is unavailable, whose journey requires a connection, or whose booked service is disrupted. Operators also need to react to bus, driver, timetable and service constraints without manually reviewing every normal booking.

- Passengers lack a single journey-oriented view of feasible services and alternatives.
- Choosing a service often requires balancing time, fare, directness, amenities and seat availability.
- A cancelled or disrupted service can require rapid re-planning for multiple affected passengers.
- Operators need visibility of service capacity, fleet resources, passenger impact and operational conflicts.
- High-impact changes require accountable human authorization, while routine recommendations and bookings should remain automatic.

3. Proposed Solution

WayPoint provides three connected capabilities: a Journey Planning Engine for deterministic feasibility and candidate generation; a passenger Booking Platform for seat reservation and ticketing; and an Operations Intelligence layer for disruption analysis and controlled operational actions.

Capability	What WayPoint Provides	Primary Benefit
Journey Planning	Origin/destination search, timetable-aware candidate journeys, direct/connecting options, preference-aware ranking, boarding-point guidance and feasibility checks.	Helps passengers choose a journey rather than simply find a bus.
Booking & Ticketing	Temporary seat holds, transactional confirmation, payment sandbox, QR	Provides a complete digital booking transaction.

	e-ticket, cancellation and refund status.	
Operational Intelligence	Disruption records, alternative-service analysis, fleet/driver feasibility, passenger-impact analysis and rebooking recommendations.	Reduces manual operational effort while keeping high-impact decisions accountable.

4. Scope and Service Strategy

The prototype will focus on selected Sri Lankan intercity and tourist corridors rather than attempting to model the entire national transport network. Example corridors include Colombo–Ella, Colombo–Kandy, Colombo–Galle, Colombo–Nuwara Eliya, Colombo–Sigiriya and Colombo–Trincomalee. The data model is designed so additional routes, stops and services can be added.

Journey-planning scope:

- Search by origin, destination, date/time and passenger count.
- Generate feasible direct and supported connecting journey options from route and timetable data.
- Rank options using deterministic factors such as arrival time, duration, fare, directness, availability and passenger preferences.
- Check transfer feasibility using scheduled arrival/departure windows and service status.
- Show boarding/drop points and optional map/location information.
- Clearly distinguish scheduled information from simulated/live operational status used by the prototype.
- Re-plan affected journeys when a service is cancelled or materially disrupted.

5. Users and Roles

Role	Responsibilities	Permissions
Passenger	Plan journeys, compare options, select seats, pay, view tickets, cancel eligible bookings and respond to rebooking recommendations.	Passenger-owned searches/bookings/tickets and permitted cancellation/rebooking actions.
Operator / Dispatcher	Manage routes, stops, services, buses, drivers, seat layouts, service status, manifests and operational disruptions.	Operational CRUD and service-management actions within assigned scope.
Transport Manager	Review high-impact operational actions and approve, reject or request revision.	Approval of ticketed-service cancellation, major timetable change and disruptive reassignment.
Administrator	Manage users, roles, operator configuration and audit access.	System administration and controlled audit access.

6. Four Primary Business Components and Individual Ownership

Each student owns one major business component. Every component includes ASP.NET Core APIs, PostgreSQL entities/migrations, React functionality, Flutter functionality, tests, Git evidence, documentation and a distinct Agentic AI responsibility. Each component will expose at least four meaningful API endpoints and at least one business-specific operation beyond basic CRUD, matching the assignment requirement.

Owner	Primary Component	Core Operations	Distinct Agentic AI Contribution
Student 1	Journey Planning & Route Catalogue	Routes, stops, tourist destinations, timetables, service listings, search and journey candidate generation.	Journey Planner Agent: decomposes the passenger objective and identifies compatible route/service candidates.
Student 2	Fleet, Seat & Resource Feasibility	Buses, seat layouts, seats, drivers, assignments, maintenance/resource status and availability.	Resource Feasibility Agent: checks replacement buses, seats, drivers, conflicts and transfer/resource constraints.
Student 3	Booking, Ticketing & Passenger Options	Seat holds, bookings, payments, tickets, cancellations/refunds and option cost evaluation.	Booking Options Agent: evaluates bookable alternatives, fare differences, seat availability and policy outcomes.
Student 4	Disruption, Rebooking & Approval	Service alerts, disruption cases, passenger impact, rebooking records, approval records and notifications.	Validation & Safety Agent: validates proposed remedies, classifies impact and enforces the approval boundary.

7. Component Scope and Minimum API Coverage

7.1 Journey Planning & Route Catalogue

- Manage routes, route stops, boarding/drop points, tourist destinations, services, fares and amenities.
- Search services by origin, destination, date/time and passenger preferences.
- Generate journey candidates and calculate deterministic feasibility/ranking inputs.
- Minimum API examples: route CRUD; service CRUD; stop lookup; journey-search endpoint; journey-candidate evaluation operation.

7.2 Fleet, Seat & Resource Feasibility

- Manage buses, seat-layout templates, seats, drivers, assignments and resource status.
- Derive seat availability from server-side holds and confirmed bookings; clients never reserve seats directly.
- Evaluate whether a replacement bus/driver/seat capacity can support a proposed service.
- Minimum API examples: bus CRUD; seat-layout CRUD; driver CRUD; availability endpoint; replacement-resource feasibility operation.

7.3 Booking, Ticketing & Passenger Options

- Create short-lived seat holds, confirm bookings after payment, issue QR e-tickets and process eligible cancellations/refund status.
- Use database transactions to prevent conflicting seat confirmation.
- Compare bookable journey options including fare and availability.
- Minimum API examples: booking CRUD/history; seat-hold operation; payment operation; ticket endpoint; cancellation/refund eligibility operation.

7.4 Disruption, Rebooking & Approval

- Record disruptions, affected services/passengers, alerts, rebooking choices and approval decisions.
- Generate and track rebooking proposals while preserving an auditable workflow.
- Only high-impact operational actions enter the manager approval state.
- Minimum API examples: disruption CRUD; alert CRUD; rebooking operation; affected-passenger lookup; approval/revision operation.

8. Journey Planning Model

WayPoint separates deterministic journey feasibility from AI recommendation. The backend remains the source of truth for route structure, timetables, fares, seat state and business rules. The Agentic AI subsystem receives structured candidate data and can plan, delegate and rank within controlled boundaries, but it cannot invent a route, alter a seat, confirm payment or perform an operational change outside an allow-listed tool.

Stage	System Responsibility
1. Passenger objective	Flutter captures origin, destination, travel date/time, passenger count and optional preferences.
2. Candidate generation	ASP.NET Core queries PostgreSQL and generates valid direct/connecting candidates from route and timetable data.
3. Agent planning	Planner Agent creates a structured plan and delegates route, resource, booking and validation tasks.
4. Controlled evaluation	Specialized agents call allow-listed tools with validated inputs and structured outputs.
5. Deterministic validation	Backend validates schemas, route/timetable consistency, seat availability, fare rules, transfer windows and permissions.
6. Recommendation	Flutter displays ranked journey options and reasons; recommendations do not change operational records.
7. Booking	Passenger selects a valid option, creates a temporary hold and completes payment; successful payment confirms the booking transactionally.
8. Disruption recovery	If an existing booking is affected, the workflow can generate replacement options and rebooking/refund outcomes.

9. Shared Agentic AI Workflow

The Agentic AI subsystem is not a generic chatbot. It implements a multi-step domain workflow that receives a travel or disruption objective, creates a structured plan, delegates tasks to four distinct agents, uses allow-listed tools, persists workflow state, applies deterministic validation and pauses high-impact actions for authorized human approval.

9.1 Four Distinct Agents

Agent	Responsibility	Input	Output / Tools
Planner / Coordinator Agent	Understands the objective and creates the execution plan; delegates work.	Structured passenger or disruption objective.	Plan with ordered steps and agent assignments.
Journey Analysis Agent	Analyses routes, stops, timetables and service	Origin, destination, date/time, constraints.	Structured candidate journey set using

	compatibility.		route/service tools.
Resource & Booking Agent	Checks fleet, seats, fare and booking feasibility.	Candidate journey IDs and passenger/booking constraints.	Feasible alternatives, seat/fare/resource evidence.
Validation & Safety Agent	Applies safety, business-rule and impact checks and determines whether approval is required.	Proposed remedy and tool results.	Validation result, impact classification and approval requirement.

9.2 Allow-Listed Tools

- SearchRoutes / SearchServices
- GetBoardingPoints / GetTimetable
- CheckTransferFeasibility
- CheckSeatAvailability / CheckReplacementResources
- CalculateFareDifference / CheckCancellationPolicy
- CreateRebookingProposal
- CalculatePassengerImpact
- RequestManagerApproval
- ApplyApprovedOperationalChange
- SendPassengerNotification

Tools will validate all inputs and return structured outputs. Tool permissions follow least privilege. The AI cannot directly access PostgreSQL, payment credentials, arbitrary URLs or unrestricted application commands.

9.3 Assessed End-to-End Agentic Workflow

1. A passenger searches in Flutter for Colombo to Ella and specifies a preference such as arrival before 6:00 PM and an optional direct-service preference.
2. ASP.NET Core authenticates the request and obtains candidate services from PostgreSQL.
3. The Planner Agent creates a structured plan and delegates route analysis, resource feasibility and validation tasks.
4. The specialized agents use allow-listed tools to evaluate candidate journeys; outputs are persisted as structured workflow state.
5. Deterministic validation checks route/timetable consistency, transfer windows, seat availability, fare rules and passenger constraints.
6. Flutter displays valid ranked recommendations. A normal recommendation requires no manager approval.
7. If the scenario is changed to a cancelled ticketed service, the workflow analyses replacement options and calculates passenger impact.
8. If the proposed remedy requires cancelling a ticketed service, materially changing a timetable or disruptively reassigning a bus/driver, the workflow enters PendingManagerApproval.
9. The Transport Manager reviews the evidence in React and can approve, reject or request revision.
10. The backend applies an approved change transactionally, records the approval decision and returns the final status to the initiating passenger in Flutter. The outcome is an auditable success or safe failure.

10. Approval Model

Event	Automatic Behaviour	Manager Approval
Journey search and recommendation	Generate and display valid options.	No
Temporary seat hold / successful	Execute automatically after	No

payment / e-ticket	deterministic validation and payment confirmation.	
Passenger cancellation allowed by policy	Calculate eligibility and process the configured workflow.	No
AI finds an existing alternative after disruption	Present/recommend the alternative without changing operational records.	No
Cancel a ticketed service	Create an auditable exception with affected passengers and impact evidence.	Yes
Major timetable change	Validate proposed change and pause before execution.	Yes
Disruptive bus/driver reassignment	Validate feasibility and passenger impact, then pause before execution.	Yes

11. React Web Application

- Role-based operator dashboard covering route/service occupancy, revenue, upcoming departures, failed payments, disruptions and journey-planning analytics.
- CRUD interfaces for routes, stops, tourist destinations, services, buses, drivers, seat layouts, amenities and fare rules.
- Search/filter/sort/pagination for services, bookings, manifests, payments, disruptions and workflow records.
- Journey-planning and disruption workbench showing candidate alternatives, resource evidence and passenger impact.
- Manager approval workbench with before/after impact, affected passenger count, validation summary and approve/reject/revise actions.
- Agent execution summary with workflow state, tool calls, validation outcomes, timings, errors, retries and final result/safe failure.
- Protected routes and role-based navigation with responsive, accessible loading, empty, success and error states.

12. Flutter Mobile Application

- Registration/login/logout with secure token storage and protected screens.
- Origin/destination/date/time/passenger-count journey search with preference filters.
- Journey recommendation cards comparing departure, arrival, duration, fare, directness, boarding point, amenities and seat availability.
- Map/location feature for boarding-point guidance where the selected third-party map service supports it.
- Interactive seat selection, payment hand-off, e-ticket wallet and QR display/scanning for verification.
- Booking history, cancellation/refund eligibility, service alerts and disruption rebooking.
- Agent recommendation submission and workflow status display when a journey or disruption requires agent analysis.
- Date/time selection and notifications as meaningful device features.

13. Third-Party Integration

The primary meaningful integration will be a payment-gateway sandbox for secure checkout. A map/location API may additionally be used for boarding-point guidance. External calls will be made through ASP.NET Core, not directly from React or Flutter. Credentials will be stored in environment variables and never committed to GitHub.

- Business purpose: provide realistic payment confirmation without storing card data in WayPoint.
- Failure handling: timeout, invalid response, duplicate callback, retry and payment failure states.
- For maps/location: handle unavailable service, invalid coordinates and rate limits.
- Only the minimum required data will be sent to third-party services.

14. Architecture and Data Design

The architecture follows the assignment's integrated-system rule: React and Flutter communicate only with the ASP.NET Core Web API; ASP.NET Core is the authoritative application layer; PostgreSQL is the authoritative relational data store; and the Agentic AI subsystem is initiated through ASP.NET Core. If a separate AI service is used, it remains internal and is never called directly by either client.

Core entities are expected to include:

- User, Role, Operator
- Route, RouteStop, BoardingPoint, TouristDestination, Service, FareRule, Amenity
- Bus, SeatLayout, Seat, Driver, Assignment, MaintenanceRecord
- Passenger, JourneySearch, JourneyCandidate
- SeatHold, Booking, Ticket, PaymentAttempt, Refund
- ServiceAlert, Disruption, RebookingProposal, ApprovalDecision
- AiWorkflow, AiWorkflowStep, AiToolCall, AiValidationResult

The database will use normalized relationships, primary/foreign keys, unique constraints, indexes for search and seat lookup, EF Core migrations, seed data, CreatedAt/UpdatedAt audit fields and transactions where required. Hidden model reasoning, passwords, tokens and unnecessary sensitive information will not be stored.

15. Security and Quality

- JWT authentication and role-based authorization.
- Password hashing and secure configuration.
- DTO validation and server-side business-rule validation.
- Global exception handling, structured logging, CORS and Swagger/OpenAPI.
- Allow-listed Agentic AI tools with validated inputs and structured outputs.
- Prompt-injection resistance, output validation, timeouts, retry limits and safe failure.
- Audit logging for high-impact actions and AI workflow execution summaries.
- Concurrency protection for seat holds and booking confirmation.
- No secrets or credentials committed to GitHub.

16. Testing, Agent Evaluation and Performance

Area	Planned Evidence
Backend	Unit, service-layer, validation, authentication/authorization, controller and API integration tests.
Database	PostgreSQL integration tests, constraints, migrations, transaction behaviour and concurrent seat-hold tests.
React	Component, form-validation, protected-route, API-integration and error-state tests.
Flutter	Unit, widget, navigation, form-validation and API-integration tests.
End-to-End	At least one Flutter → ASP.NET Core → PostgreSQL → Agentic AI → React approval → Flutter status workflow.
Agent Evaluation	Golden cases, structured-plan assertions, distinct-agent participation, tool-selection checks, schema validation, deterministic business rules, approval enforcement, prompt-injection resistance, failure recovery and safe-failure tests.
Performance	Concurrent requests, response time, success/failure rate, database response and Agentic AI latency.

LLM-as-a-judge may be used only as supporting evidence. Rule-based assertions, schema validation, golden cases, deterministic validators and human review will provide the core Agentic AI evaluation evidence.

17. Git, CI/CD and Collaborative Development

- GitHub repository created from the beginning of the project.
- Feature branches, meaningful commits, issues, pull requests, reviews and a project board.
- GitHub Actions CI that restores, builds and runs backend automated tests on every push and pull request to main.
- Additional frontend build/lint, Flutter analysis and deployment checks where practical.
- Regular technical contribution by all four students across backend, database, React, Flutter, AI and testing.
- Contribution evidence maintained through Git history, PRs, issue ownership and test evidence.

18. Architecture Decision Records

The group will maintain ADRs documenting the context, options, decision and consequences for key technical choices. At minimum: React state management, Flutter state management, Agentic AI framework/orchestration, AI workflow state persistence/database strategy and cloud deployment platform. Three to six focused ADRs are planned.

19. Deployment and Documentation

- ASP.NET Core API deployed to a suitable cloud platform with working health and Swagger URLs.
- PostgreSQL deployed securely with migrations, restricted credentials and initialization instructions.
- React deployed as a working live web application configured for the deployed API.
- Flutter delivered as a runnable Android APK.
- Agentic AI deployed or run locally with complete setup requirements and startup order.

- README covering architecture, database, API, installation, environment variables, tests, deployment, live URLs, test accounts, security and individual contributions.
- One consolidated report containing group and individual sections, technical report, testing report, Agentic AI evaluation, performance report, deployment report, ADRs, diagrams, references and AI-use declarations.

20. Expected Demonstration

The demonstration will show the system as a complete journey-planning product rather than only a booking application.

11. Passenger logs into Flutter and searches Colombo → Ella for a chosen date/time with preferences.
12. WayPoint returns feasible direct/connecting journey options with fare, duration, arrival time, boarding point, amenities and seat availability.
13. The passenger selects an option, selects a seat, completes sandbox payment and receives a QR e-ticket.
14. In React, an operator marks the selected service as disrupted/cancelled.
15. The Agentic AI workflow creates a structured plan, delegates to the four distinct agents, checks alternatives/resources/booking impact and validates the proposed remedy.
16. For an ordinary alternative, the passenger receives a recommendation without manager approval.
17. For a high-impact operational change, the workflow pauses in PendingManagerApproval and the Transport Manager reviews it in React.
18. The manager approves/rejects/revises; the backend applies an approved action transactionally and records the audit outcome.
19. Flutter receives the updated rebooking/refund/ticket status.
20. A second search demonstrates the same journey-planning architecture on a high-demand corridor such as Colombo → Kandy.

21. Assignment Compliance Matrix

SE3090 Requirement	WayPoint Response
ASP.NET Core Web API	Authoritative public backend for clients, business rules, security, workflow initiation and approval.
PostgreSQL + EF Core	Normalized relational database with migrations, constraints, indexes, seed data and transactions.
React	Operator/admin/manager workspace with CRUD, dashboards, reporting, AI monitoring and approvals.
Flutter	Passenger journey-planning, booking, ticketing, alerts, rebooking and device features.
Four major components	Journey Planning & Route Catalogue; Fleet/Seat/Resource Feasibility; Booking/Ticketing; Disruption/Rebooking/Approval.
Every student across required stack	Each owner implements backend/database/React/Flutter/tests/Git/documentation plus distinct AI work.
At least four API endpoints/component	Each component explicitly plans ≥4 meaningful endpoints plus a business-specific operation.
At least three roles	Passenger, Operator/Dispatcher, Transport Manager, Administrator.
CRUD + workflows + search/filter/sort/pagination/reporting	Included across route, service, fleet, booking, disruption and management modules.
Meaningful third-party service	Payment sandbox; optional map/location service.
Agentic AI Level 4 requirements	Structured planning, four distinct agents, controlled tools,

	persisted state, deterministic validation, human approval, observability and safe failure.
Integrated cross-platform workflow	Flutter → ASP.NET Core → PostgreSQL/AI → React approval → ASP.NET Core → Flutter status.
Testing	Backend, database, React, Flutter, E2E, performance and Agentic AI evaluation.
Git/CI	GitHub workflow, PRs/reviews, GitHub Actions CI and individual evidence.
Deployment	Cloud API, PostgreSQL, live React, Flutter APK and AI setup/access information.
ADR/documentation	Required architecture decisions, README and consolidated report evidence.

22. Feasibility and Scope Control

The project is deliberately scoped as a prototype for selected corridors and a controlled company-operated transport network. It will not attempt to integrate every Sri Lankan bus operator, guarantee real-time national traffic data or automate irreversible operational decisions. Live vehicle tracking can be represented with controlled/simulated status data for the prototype. This keeps the project achievable within the assignment period while preserving a credible path to future real-world expansion.

- Phase 1: shared identity, database, core API, route/timetable data and basic React/Flutter shells.
- Phase 2: journey-planning engine, fleet/seat management and booking transaction.
- Phase 3: Agentic AI orchestration, four agents, controlled tools and persisted workflow state.
- Phase 4: disruption/rebooking, approval workflow, third-party integration and notifications.
- Phase 5: testing, performance evaluation, deployment, CI evidence, ADRs and demonstration preparation.

23. Conclusion

WayPoint is positioned as a practical intercity journey-planning solution with booking and transport operations built around the same shared platform. Its key improvement over a conventional booking application is that the system plans and re-plans journeys using structured transport data, while its Agentic AI subsystem coordinates multi-step analysis under deterministic business controls. Routine passenger actions remain automatic; only high-impact operational changes require authorized human approval.

The resulting scope directly supports the SE3090 assessment: four technically owned business components, a shared ASP.NET Core/PostgreSQL foundation, meaningful React and Flutter applications, a four-agent Agentic AI workflow, third-party integration, testing, CI/CD, deployment, auditability and a complete cross-platform demonstration.

24. References and Evidence Basis

Primary assignment basis: SE3090 – Software Engineering Frameworks, Assignment 1 – Integrated Full-Stack and Agentic AI Application Development, 2026 specification and marking scheme. The proposal preserves the assignment terminology and requirements for the mandatory technology stack, four-student component ownership, Agentic AI minimum acceptance workflow, integrated architecture, third-party integration, testing, CI/CD, deployment, ADRs, documentation and evaluation.

The original WayPoint proposal was revised to make journey planning the central product capability while retaining the original booking, operations and approval concepts that already align with the assignment.